麦当劳中国 MCP · 第三方客户端

开发者手册

命令行参数 · 配置项 · 自动化集成 · 二次开发

文档版本 1.0 | 适配 MCP 服务 1.0.9（2026-09-10） | 2026-11-01

0 快速开始

三条命令跑通。普通用户请看《使用指南》，本文是给动手的人看的。

① 拿 Token

打开 **open.mcd.cn/mcp**
手机号登录 → 控制台 → 激活
复制那串 Token

② 装（其实不用装）

只要有 Node 18+
没有 npm install
没有构建步骤

③ 登录

`mcd login <token>`
`mcd home` 看一眼
就能用了

```
git clone <本仓库> && cd mcd-skill
node scripts/mcd.mjs login 你的Token
node scripts/mcd.mjs home

# 设一次偏好，以后不用再说
node scripts/mcd.mjs profile set city 上海
node scripts/mcd.mjs stores                 # 查门店
node scripts/mcd.mjs profile use-store 1    # 认准一家

# 点一单
node scripts/mcd.mjs price --item "巨无霸:1"
node scripts/mcd.mjs order --item "巨无霸:1" --confirm
```

三条不能忘的事

1. **钱永远你自己付。**程序只生成订单和支付链接，不代付、不碰支付凭证。
2. **下单要加 `--confirm`。**不加就只算价、不下单——这是故意的护栏。
3. **包内没有任何个人信息。**门店、地址、电话、备注全存在你自己电脑的 `%APPDATA%\mcd-mcp\config.json` 里，不随仓库分发。

1 接入

1.1 Token 怎么拿

1. 打开 **https://open.mcd.cn/mcp**，右上角【登录】，手机号 + 验证码。
2. 登录后右上角变成【控制台】→ 点【激活】→ 同意《使用条款》。

3. Token 生成后一键复制。

> **Token 是什么**
>
> 它只代表「你有权调用麦当劳 MCP 服务」，**不是账号密码**，不能用来登录麦当劳 App、不能付款。 同一个账号的订单/积分/券，是靠 Token 背后的手机号关联的。

1.2 三种接法，按你的环境选一种

场景	怎么做	说明
A. 用命令行 推荐，任何环境都行	`node scripts/mcd.mjs <命令>`	不依赖任何客户端。AI agent 有 shell 就能用；人自己也能用。
B. 原生 MCP 客户端支持 MCP 时	填一段 JSON（见下）	36 个工具直接进模型工具列表。代价：每轮对话都背着 36 份 schema。
C. 直接问 AI 把 SKILL.md 交给它	把 `SKILL.md` 丢给 agent	agent 读完就知道先查什么、什么不能做。

B 方式的配置：

```
{
  "mcpServers": {
    "mcd-mcp": {
      "type": "streamablehttp",
      "url": "https://mcp.mcd.cn",
      "headers": { "Authorization": "Bearer 你的Token" }
    }
  }
}
```

1.3 自检

```
node scripts/mcd.mjs status
```

正常会打印：连接正常、协议版本 `2025-06-18`、工具数量 `36`。

报 **401** = Token 错或过期；报 **429** = 一分钟超过 600 次请求，等一会儿。

2 第一次配置：让他记住你

这一步只用做一次。做完之后，每条命令都不用再重复「哪家店、送到哪、要不要餐具」。

2.1 城市和门店

```
node scripts/mcd.mjs profile set city 上海
node scripts/mcd.mjs profile set keyword 中山北路3663号    # 找店用的地标，随便写个附近的地方
node scripts/mcd.mjs stores                                 # 出来 5 家，带编号
node scripts/mcd.mjs profile use-store 1                    # 第 1 家设为默认
```

> **麦当劳 MCP 不支持 GPS 定位**
>
> 所以「附近的店」必须靠你给一个**文字地址或地标**，程序没法自己知道你在哪。

2.2 收货地址、联系人、电话

点外卖（麦乐送）必须有收货地址。有两种用法：

用法一：用账号里已经存过的地址（最省事）

```
node scripts/mcd.mjs address          # 列出账号里全部地址，带编号
node scripts/mcd.mjs address use 3    # 第 3 个设为默认

node scripts/mcd.mjs price --mode delivery --item "巨无霸:1"
```

用法二：新建一个地址

```
node scripts/mcd.mjs address new \
  --name 张三 \
  --phone 13800138000 \
  --city 上海 \
  --addr 中山北路3663号 \
  --detail "12号楼301室" \
  --confirm
```

参数	含义	备注
`--name`	收货人姓名	可以只写姓
`--phone`	手机号	**必须 11 位纯数字**，程序会校验
`--city`	城市	如「上海」
`--addr`	街道地址	路名门牌
`--detail`	门牌细节	楼栋/单元/房间，骑手靠这个找到你
`--gender`	称谓	选填，「先生」「女士」

电话会不会被记下来?

建地址时电话**必须发给麦当劳**（不然骑手联系不到你）。但**本地不会把它抄进配置文件**—— 之后从接口拉回来的地址列表里，电话本来就是**打码**的（`138****5678` 这种）。

2.3 备注（口味、配送要求）

```
node scripts/mcd.mjs remark "不要生菜，可乐去冰"
node scripts/mcd.mjs remark                          # 看当前备注
node scripts/mcd.mjs remark clear                    # 清掉
```

什么时候生效	说明
麦乐送外送单	作为订单的 `remark` 字段真实传给门店和骑手，**上限 50 字**，超了自动截断。
到店自取单	接口没有备注字段，**不会**传给厨房。备注只用来提醒你自己。

「少冰」「不要番茄酱」这类

如果某样餐品支持「特制」（去冰、不加酱），要在 `query-meal-detail` 里拿到它自己的特制编码才能生效， 不是所有商品都支持。备注字段只是**给门店看的一句话**，不是点单选项。

2.4 其余偏好一览

命令	作用
`profile set mode delivery`	默认取餐方式：`pickup` 到店 / `dt` 得来速 / `delivery` 麦乐送
`profile set taste.tableware true`	默认要餐具
`profile set budget 30`	单餐预算，超了只提示、不拦
`profile set autoCoupon false`	关掉「算价时自动挑最省的券」
`profile`	打印全部偏好
`profile clear`	清空全部偏好（Token 保留）

3 点一单

3.1 到店自取（最常用）

```
node scripts/mcd.mjs menu --grep 巨无霸
node scripts/mcd.mjs price --item "巨无霸:1" --item "中薯条:1"
node scripts/mcd.mjs order --item "巨无霸:1" --item "中薯条:1" --confirm
```

`price` 会自动完成：拉菜单（缓存 10 分钟）→ 把「巨无霸」解析成商品编码 → 查本店可用券 → 逐张试价 → 挑最省的 → 算总价。名字不用写全，写编码也行。

3.2 麦乐送（外卖）

```
node scripts/mcd.mjs price --mode delivery --item "巨无霸:1"
node scripts/mcd.mjs order --mode delivery --item "巨无霸:1" --confirm
```

3.3 预约（指定时间送到/取）

```
node scripts/mcd.mjs price --item "巨无霸:1" --at "2026-11-02 18:00"
```

3.4 再来一单

```
node scripts/mcd.mjs history 5     # 看最近几单
node scripts/mcd.mjs reorder       # 上一单按当前门店重新算价
node scripts/mcd.mjs reorder --confirm
```

> **下单前一定要核对四样**
>
> **门店 • 商品与数量 • 实付金额 • 取餐/送达时间**。金额单位是「元」，程序已经换算好了。

3.5 套餐：先看能换什么，再报价

点套餐（名字带「三件套 / 四件套 / 组合 / 随心配」）时多走一步：

```
node scripts/mcd.mjs detail "巨无霸四件套"
```

它会列出这套餐由几部分组成、每部分默认是什么、**还能换成什么、各加多少钱**：

```
组成 4 项：
2) 小食 ×1
    ★ 中薯条
    可换成（8 个）：
      · 蓝莓爆爆珠麦旋风          + ¥1
      · 奥利奥麦旋风              + ¥2
      · 大杯玉米杯                + ¥2.5
4) 选择套餐内饮料 ×1
    ★ 可乐中杯
    可换成（29 个）：雪碧 +¥0 / 拿铁 +¥11 …
```

能不能直接改？——不能

实测：把替换写成 `modification` 传进算价接口，价格和商品清单**完全不变**。 接口只提供「看」，没有「在这个位置换那个」的字段。

想换只有两条路：① 去麦当劳 App 里改；② 按**单点重组**（汉堡 + 单点小食 + 单点饮料），价格重新算。

3.6 每次变动都要重报总价

商品、规格、数量、券、取餐方式**任何一项变了**，都要重新 `price`， 并明确说出新的实付金额和「比上一版多/少 ¥X」。永远不要沿用上一轮的金额。

同一份套餐的三种价	例子
菜单标价（浏览时看到的）	¥29
算价（`calculate-price`，单位是分）	¥58.00
实付（下单后 `realTotalAmount`）	¥33.50

这三个数可以都不一样：到店价、外送价、活动价、券、配送费、包装费各算各的。 **报价只用 `price` 的「▶ 实付」**；菜单价和算价不一致时程序会打 ⚠️， 照实转述，别只挑一个数字报。

4 钱怎么付（重点）

4.1 它给的不是「付款码」，是一个支付链接

下单成功后，麦当劳返回三个东西：

字段	是什么	怎么用
`orderId`	订单号	查进度、取消、找客服
`payH5Url`	**支付页链接（H5）**	点开 → 麦当劳自己的收银台 → 选支付方式 → 付钱
`pickupCode` `lockerQrCode`	取餐码 / 取餐柜二维码	**取餐用的，不是付款用的**，别搞混

付款码 vs 取餐码

付款码=收银台弹出的、给钱包 App 扫的码（在 `payH5Url` 那个页面里）。
取餐码=付完钱、餐做好了给你领餐用的号，和钱无关。

4.2 支持微信还是支付宝?

程序不决定，麦当劳的收银台决定。 MCP 把所有支付相关的事都收敛成一个 `payH5Url`： 你在浏览器/微信里打开它，页面里提供哪些支付渠道（微信支付、支付宝、云闪付、银行卡等）， 由麦当劳根据设备、场景、地区实时决定。接口层**没有**「支付方式」这个字段，也不返回支付二维码图片。

所以正确的说法是

它**不挑支付方式、也不限制支付方式**——它把收银台交给你，你在里面怎么付都行。 这也是它安全的地方：客户端全程碰不到你的支付凭证。

说明：以上来自接口返回字段与官方工具描述。真实订单的支付页具体呈现（有没有支付宝、能不能用余额） 需要下第一单才能确认——本文档不把未验证的事写成结论。

4.3 支付链接会怎么到你手上

谁在操作	形式
你自己用命令行	终端里直接打印 `payH5Url`，复制到浏览器打开
AI agent 帮你点	agent 把链接原样贴回对话里
聊天机器人帮你点	它把链接单独一行发到 IM 里，你在手机上点开付款

IM 里发链接的两点注意

① 链接很长，容易被折叠，让机器人**单独一行发**；② 只在私聊里点餐，群里发等于把付款入口摊给所有人。

4.4 付完之后

```
node scripts/mcd.mjs order-status <orderId>
```

订单状态：1 待支付 → 2 配餐中（已支付）→ 4 配送中 → 6 已完成。 支付页还有一个 expirePayTime，超时订单会自己取消。

5 券、积分、省钱

```
node scripts/mcd.mjs home             # 积分 + 本月将过期 + 可领券 + 上一单
node scripts/mcd.mjs coupons          # 卡包里的券
node scripts/mcd.mjs claimable        # 麦麦省里「可领」的券
node scripts/mcd.mjs claim --confirm # 一键全领
node scripts/mcd.mjs store-coupons    # 当前门店这张单能用的券
node scripts/mcd.mjs points           # 积分明细
```

命令 / 工具	含义
`query-my-coupons`	卡包里有什么券
`query-store-coupons`	**这张单、这家店**到底哪张能用（两者不能互相替代）
`available-coupons` / `claimable`	麦麦省里**可领**的券
`auto-bind-coupons` / `claim`	一键领光所有能领的券（**不可逆，要 `--confirm`**）

`price` 默认会**自动挑最省的券**：把每张券在每件商品上试一遍，取实付最低的组合。 不想要就加 `--no-auto-coupon`。

5.1 可领券的自动提醒

`price` / `order` 每次都会顺手查一次可领券，有就在末尾提一句：

> 💡 还有 3 张券可以领：免费脆薯饼 / 13.5元中薯条买一送一 / 19.9元王牌小食任选2
> 领了再算一次可能更省：`node scripts/mcd.mjs claim --confirm`

- 结果缓存 5 分钟，不会每个命令都去打接口；
- `--no-claim-hint` 可以关掉；`autoCoupon` 关着时也不提示；
- `available-coupons` 有时返回 JSON、有时返回 Markdown，解析器两种都认；
- 查不到就静默跳过 —— 领券提示失败不该阻塞算价。

6 查单与取消

```
node scripts/mcd.mjs history 5              # 最近 5 单
node scripts/mcd.mjs order-status <orderId>
node scripts/mcd.mjs cancel <orderId> --reason 3 --confirm
```

取消原因码：1 改主意了 · 2 重复下单 · 3 点错/点多/点少 · 4 地址电话填错 · 5 时间选错 · -1 其它。

下单超时了别急着重试

先 `history` 看有没有已经生成，确认没有再重下。重复下单是要扣两次钱的。

6.1 主动推送：配餐中 → 配送中 → 已送达

先说清楚：装上不等于会推送

MCP **没有任何回调或订阅机制**，只能靠反复查 `query-order`。 本工具提供的是「轮询 + 判定变化 + 执行回调」，**发到哪必须你自己接。** 没有接回调、也没有进程在跑的时候，不会有任何通知。

完整链路是三段，缺一段都不响：

```
计划任务/常驻进程  →  mcd watch --once  →  状态变了  →  你的通知命令  →  QQ/TG/邮件
```

```
# 前台盯：每 60 秒查一次，只有状态变了才打印
node scripts/mcd.mjs watch <orderId> --interval 60

# 交给计划任务 / cron：每次只查一次，状态变了才触发回调
node scripts/mcd.mjs watch --last --once --quiet --on-change "你的通知程序"
```

参数 / 占位符	说明
`--last`	盯「刚刚下的那一单」（读本地下单缓存），计划任务不用传订单号
`--max-age 180`	最近一单超过 180 分钟就退出，不无限轮询死单
`{status}` / `{orderId}` / `{code}` / `{amount}`	回调命令里的占位符
`profile set notify "…"`	把回调存进偏好，`watch` 不带 `--on-change` 时自动用它

回调里传中文：用环境变量，别用参数

`runHook` 同时把值放进 `MCD_STATUS` / `MCD_ORDER_ID` / `MCD_CODE` / `MCD_AMOUNT`。Windows 上经 shell 传中文参数会被转码搞坏， **回调程序读 `process.env.MCD_STATUS` 最稳。**

为什么可以「每 2 分钟跑一次」

`--once` 会把上次状态记进本地缓存（24 小时），**同一状态不会重复通知**； 已经收尾（完成/取消）的单下次直接短路，连接口都不打。所以丢进计划任务里是安全的。

Windows 计划任务（用 PowerShell 注册，别用 `schtasks` 命令行——引号会被吃掉）：

```
$action  = New-ScheduledTaskAction -Execute "wscript.exe" -Argument '"<绝对路径>\mcd-watch.vbs"'
$trigger = New-ScheduledTaskTrigger -Once -At (Get-Date) -RepetitionInterval (New-TimeSpan -Minutes 2) -RepetitionDuration (New-TimeSpan -Days 3650)
Register-ScheduledTask -TaskName "MCD Order Watch" -Action $action -Trigger $trigger -Force
```

`mcd-watch.vbs`（**必须纯 ASCII**，wscript 按 ANSI/GBK 读）里只有一行：

```
Set sh = CreateObject("WScript.Shell")
sh.Run "cmd /c node ""<仓库>\scripts\mcd.mjs"" watch --last --once --quiet", 0, False
```

Linux / macOS 就一行 cron：

```
*/2 * * * * cd /path/to/mcd-skill && node scripts/mcd.mjs watch --last --once --quiet >> watch.log 2>&1
```

6.2 接任意 bot：notify（不用写代码）

`scripts/notify.mjs` 是通用分发器，目标写在配置里，三种出口：

type	关键字段	说明
`webhook`	`url`（必填）/ `method` / `headers` / `body`	任意 HTTP 接口：QQ、飞书、钉钉、Slack、Bark、自建服务
`telegram`	`token` / `chatId`	Telegram Bot API
`command`	`cmd`	任意命令行程序

```
node scripts/mcd.mjs notify add --type webhook --url "https://你的机器人回调地址"
node scripts/mcd.mjs notify test        # 立刻发一条，验证通不通
node scripts/mcd.mjs notify list        # 看当前目标
node scripts/mcd.mjs notify remove 2    # 删掉第 2 个
node scripts/mcd.mjs notify text "🍟 {status}！"
```

第一次 `notify add` 会**自动**把 `profile.notify` 设成 `node "<仓库>/scripts/notify.mjs"`，之后 `watch --last --once` 不用再带 `--on-change`。

模板：`{{text}}` = 已 JSON 转义的整段文案；`{status}` `{orderId}` `{code}` `{amount}` 原样替换（URL 里也能用）。默认请求体是 `{"text":"…"}`，对接别家自己给 `--body`：

```
# 飞书
--body '{"msg_type":"text","content":{"text":"{{text}}"}}'
# 钉钉
--body '{"msgtype":"text","text":{"content":"{{text}}"}}'
# Bark（iOS），URL 里直接带内容
--method GET --url "https://api.day.app/你的key/麦当劳-{status}"
```

写自测脚本时的一个坑

用 `spawnSync` 调 CLI 会把自身事件循环堵住，你起的本地 HTTP 服务就没法响应， 测试必然超时。用异步 `execFile/spawn`。这不是产品 bug。

最后一道门：不想用 notify.mjs，就 `--on-change "你的程序"`， 它会继承 `MCD_STATUS` / `MCD_ORDER_ID` / `MCD_CODE` / `MCD_AMOUNT` / `MCD_TEXT`，不限语言不限平台。

7 隐私：你的信息去哪了

7.1 包里有你的个人信息吗

没有。分发包是「零个人信息」的

仓库里只有代码和文档，示例一律用「张三」「13800138000」「人民广场」这类虚构值。 你的门店、地址、电话、备注、Token 全部在**你自己电脑**上，不在包里。

7.2 本地到底存了什么、存在哪

文件	内容	去不去仓库
`%APPDATA%\mcd-mcp\config.json` Linux/macOS：`~/.config/mcd-mcp/`	Token、门店、地址、备注、预算	**不会。**它在系统配置目录，不在项目目录里；`.gitignore` 另外还挡了一层
`…\mcd-mcp\cache\`	菜单、门店列表缓存（10 分钟过期）	**不会**
仓库目录	只有代码、文档、生成的工具清单	会（这就是你要分享的部分）

配置文件写的时候是 **600 权限**（只有你自己能读）。想彻底清掉：`mcd logout` 删 Token， 手动删掉整个 `mcd-mcp` 目录即全部清空。

7.3 什么信息会被发出去

你的电脑
命令 + 本地偏好
（只在这一侧）

→

mcp.mcd.cn
只收：Token、门店码、商品码、地址 ID、手机号（建地址时）

→

麦当劳账户
订单、积分、券、地址

- **收件人只有麦当劳。**本程序不往任何第三方服务器发数据，没有埋点、没有遥测、没有统计上报。
- **不收集设备信息、不读通讯录、不读相册、不定位。**
- **手机号只在建地址时发一次**，之后接口回传的都是打码值。

7.4 用 AI / 聊天机器人点餐时的额外风险

这一层不是本 skill 引入的，但必须说清楚：

风险点	实际情况	怎么规避
AI 读到你的地址	agent 调 `address` 时，地址会进它的上下文	没有地址信息给它是调不出结果的；这是功能前提，不是泄漏
对话被模型服务商看到	无论用哪家 AI，上下文都会发给模型厂商	别在聊天里报完整手机号；用 `address use` 选编号，别贴全文
聊天记录留痕	QQ 机器人会把对话存进本地会话库	私聊里点餐；别在群里发地址和支付链接
Token 被看到	`status` 已打码；但有人能读你屏幕就另说	别把 Token 贴进聊天、截图、issue

一句话

本 skill 本身**不新增任何泄漏面**——它不发数据、不写日志、不把个人信息系统外传。 真正会看到你信息的是**你选的 AI 服务商**和**麦当劳**。选哪家 AI，等于选谁看你的地址。

8 交给 AI：它应该怎么用

1. **先设偏好再看菜单。**没有默认门店就问，不要猜。
2. **不可逆动作必须确认。**下单、取消、领券、抽奖、扣积分、建地址——报清楚「门店/商品/实付/时间」再等一句「确认」。
3. **金额单位。**`calculate-price` 返回的是**分**；`price` 命令已经换成元，直接用。
4. **超时先查单**，别重下。
5. **支付链接原样给**，不代付、不索取验证码。

完整规则见仓库里的 `SKILL.md`；接聊天机器人见 `references/bot-integration.md`（可选）。

9 附录

9.1 排错

现象	原因 / 处理
`HTTP 401`	Token 失效，去开放平台重新复制，再 `login`
`HTTP 429`	超 600 次/分钟，等一分钟
「还没有默认门店」	`stores` → `profile use-store 1`
「匹配到 N 个商品」	名字写具体点，或直接给商品编码
算价对不上	券不适配本店/本商品，用 `store-coupons` 复核
菜单里没有某样	时段问题（早餐只在早市），或这家店不卖
PowerShell 里 JSON 报错	别手写 JSON，用 `k=v` 或 `--file`
提示「已拦截，没有发出」	写入类命令缺 `--confirm`，核对入参后再加

9.2 命令总表

命令	作用
`login` / `logout` / `status` / `where`	Token 与连接
`home`	积分 / 券 / 默认门店 / 上一单
`profile` (show/set/unset/clear/use-store/use-address)	点餐偏好
`address` (list/new/use)	收货地址
`remark`	口味 / 配送备注
`stores` / `menu` (`--grep` `--cat` `--all`)	找店、看菜单
`detail`	套餐组成、可替换项与差价
`price` (`--detail` `--brief`) / `order`	算价与下单
`history` / `reorder` / `order-status` / `watch` / `cancel`	订单管理、盯单、取消
`coupons` / `store-coupons` / `points`	券与积分
`call` / `tools` / `schema` / `doc`	裸调用与工具清单

9.3 边界

- 本项目是第三方客户端，**与麦当劳中国无隶属关系**，不构成官方背书。

- 价格、库存、活动、支付渠道以麦当劳官方页面与实际扣款为准。
- 下单、付款、退款的责任在用户本人。
- 服务仅面向中国大陆（不含港澳台）。

 | 代码 MIT，文档 CC BY 4.0